

AI 协作反思日志 #4 - Phase 4 核心开发与功能完善

团队与日期

项目	内容
团队	辛巴巴把鹿比啦
阶段	P4 编码开发
日期	2026-06-07
参考材料	Sprint 合并任务看板、AI 代码信任度实验、AI 调试对决、Bug 修复日志、CI/CD 配置与运行记录

1. 本阶段 AI 使用清单

任务	AI 工具	Prompt 摘要	AI 输出质量(1-5)	人工修改幅度(%)
P4-002 Sprint 合并任务看板	Codex	根据 P4 交付要求拆分 P0/P1/P2、测试、文档任务，补充负责人、工时、依赖和完成标准	4	25
P4-005 接口契约整理	Codex	对照 P3 API 规范和前端页面，整理页面-接口-字段对应关系	4	30
P4-010 至 P4-024 核心功能开发	Codex/Claude	基于 Spring Boot、Vue 3 和现有模块结构补齐认证、任务、审核、提交、成长、通知等功能	3	30
P4-030 单元测试	Codex	为核心服务生成测试用例，覆盖权限、状态流转、异常输入和边界条件	4	35
P4-031 集成测试	Codex	设计注册、登录、仓库同步、任务发布、接取、提交、审核的端到端接口测试	4	30
P4-032 CI/CD	Codex	生成 GitLab CI 与 GitHub Actions 配置，覆盖静态检查、前端构建、后端 verify 和产物保存	4	20
P4-033 AI 代码	Codex	让 AI 直接生成 <code>AdminQuestServiceImpl.reviewQuest</code> ，再用人工审查和测试验证可靠性	2	60

任务	AI 工具	Prompt 摘要	AI 输出质量(1-5)	人工修改幅度(%)
信任度实验				
P4-034 AI 调试对决	Codex	将真实 Bug 的现象、复现步骤和相关代码交给 AI，比较纯人工与 AI 辅助定位效果	3	40
P4-035 Bug 修复日志	Codex	为 P4 联调 Bug 生成记录模板并补充现象、根因、修复方案和验证结果	4	25
P4-037 演示说明	Codex	按课程要求整理本地运行方式、演示账号、测试数据和核心演示路径	4	20

说明：人工修改幅度为团队复盘估算值，衡量的是 AI 初稿进入最终交付物前需要人工调整的比例。

2. AI 助力分析

2.1 AI 在编码中最有价值的场景

AI 在本阶段最有价值的编码场景是“已有清晰模块边界和接口契约时的样板代码补齐”。例如后端 Controller、DTO、Repository 查询方法、Service 基本分支，以及前端 API 封装、路由接入、状态提示等，AI 能快速按照现有目录结构生成可编译的初稿。

2.2 AI 在测试生成中最有价值的场景

AI 对测试最有帮助的是补充边界用例清单。团队把状态流转规则、权限矩阵和异常码告诉 AI 后，AI 能较快给出测试结构，例如：

- 非管理员不能审核任务。
- 非维护者不能发布任务。
- PR 未合并时不能审核通过成果。
- 同一任务不能被重复接取。
- 任务下架后需要处理仍处于 ACTIVE 的接取记录。

2.3 AI 在调试中最有价值的场景

AI 在局部、现象明确的 Bug 上效率较高。比如 BUG-002“仓库同步成功但 Issue 列表为空”，只要提供同步方法、Gitea adapter 和数据库表状态，AI 很快能指出同步接口只标记了 SYNCED，没有拉取并 upsert Issue。

3. AI 误导分析

3.1 “AI 代码信任度实验”数据摘要

本阶段选择 `AdminQuestServiceImpl.reviewQuest` 作为 AI 代码信任度实验对象。该功能涉及管理员权限、任务状态流转、审核记录落库、下架联动接取记录，适合观察 AI 是否真正理解业务约束。

指标	AI 直出	人工审查修复后
编译是否通过	通过	通过
功能是否可运行	主路径可运行，但权限、状态和下架联动不可信	正常审核、驳回、下架、权限错误、状态错误均有处理
测试是否通过	未通过， <code>Tests run: 106, Failures: 4, Errors: 0, Skipped: 7</code>	通过， <code>Tests run: 106, Failures: 0, Errors: 0, Skipped: 7</code>
主要问题	缺少 ADMIN 校验；缺少状态机约束；下架不取消 ACTIVE 接取记录；默认异常不符合 API 规范	使用 <code>BusinessException</code> 和错误码；通过领域方法约束状态流转；下架时联动处理接取记录

数据摘要：

- AI 直出的测试通过率：约 96.2%，即 106 个测试中 102 个通过、4 个失败。
- 人工审查修复后的测试通过率：100%，即 106 个测试全部通过。
- 发现的主要问题类型：权限遗漏、状态机规则遗漏、跨表一致性遗漏、异常响应不规范、绕过领域对象方法。

这说明“能编译”和“多数测试通过”不能等价于“核心业务逻辑可信”。对 Git Guild 这类任务流转系统，人工必须逐条核对角色权限、状态前置条件和跨表副作用。

3.2 “AI 调试对决”实验总结

Bug #	Bug 描述	纯人工定位耗时	AI 辅助定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
BUG-002	仓库同步未拉取 Issue	约 10 分钟	约 2 分钟	准确	可用，且补充了 <code>upsert</code> 和状态映射细节	AI 方案 + 人工对齐契约
BUG-006	Gitea PR 合并后审核台未	约 18 分钟	首轮约 3 分钟但偏表	首轮不准确，补充	首轮不可用，补充上下文后可用	人工根因 + AI

Bug #	Bug 描述	纯人工定位耗时	AI 辅助定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
	同步本地 PR 状态		象；补充上下文后准确	上下文后准确		补充实现
BUG-001	重复导入同一 Gitea 仓库生成多条记录	约 9 分钟	约 2 分钟	准确	基本可用，但遗漏“取最早记录”和数据库唯一约束边界	AI 骨架 + 人工补边界

AI 擅长调试的 Bug 类型：

- 现象明确、根因局部、可通过单个方法静态推理的问题。
- 后端已返回错误码，但前端没有展示或没有处理的问题。
- 缺少查询、upsert、状态映射等常见数据同步步骤的问题。

AI 不擅长调试的 Bug 类型：

- 需要跨多个角色路径推理的时序问题。
- 同一份状态在不同页面、不同接口间刷新策略不一致的问题。
- 涉及“谁负责触发同步”的架构边界问题。
- 需要同时考虑已有脏数据、幂等、唯一约束和历史记录迁移的问题。

AI 修复方案的典型问题：

- 倾向补前端按钮或提示，容易绕过后端根因。
- 倾向只修当前复现样例，遗漏已有历史数据。
- 倾向在 Service 中直接改字段，绕过领域对象方法。
- 不主动核对 P3 API 规范和数据库设计中的错误码、唯一约束、状态机。

4. Prompt 改进计划

4.1 上阶段改进措施的验证结果

P3 阶段已经要求 AI 输出必须结合类图、API 规范和数据库设计。本阶段验证结果是：当 prompt 明确提供这些上下文时，AI 生成的代码结构和命名更贴合项目；但如果 prompt 只描述功能目标，AI 仍会忽略权限、状态机和异常码。

改进有效的部分：

- 让 AI 遵守已有目录结构，减少新建不必要抽象。
- 让 AI 根据 P3 API 规范补 DTO 和接口字段。

- 让 AI 根据测试失败信息定位局部 Bug。

仍然不足的部分：

- prompt 没有强制要求列出状态前置条件时，AI 会默认状态都可转换。
- prompt 没有给出所有调用路径时，AI 会把同步逻辑假设为“别处已经发生”。
- prompt 没有要求“先说风险再给代码”时，AI 倾向直接产出看似完整的实现。

4.2 编码阶段 Prompt 最佳实践

本阶段沉淀出的编码 prompt 最佳实践：

请先阅读以下上下文，再给出最小范围修改：

1. 相关 **P3 API** 契约，包括路径、请求字段、响应字段、错误码。
2. 相关实体状态机，包括允许的状态转换和禁止场景。
3. 当前模块目录结构和已有领域方法，优先复用已有方法。
4. 权限矩阵，说明哪些角色可以操作，哪些角色必须被拒绝。
5. 需要覆盖的测试场景，包括正常流、边界流和异常流。

输出顺序：

1. 先列风险点和需要人工确认的假设。
2. 再列修改文件。
3. 最后给代码或补丁。

不要绕过现有领域对象方法，不要直接修改状态字段。

4.3 调试阶段 Prompt 最佳实践

本阶段沉淀出的调试 prompt 最佳实践：

这是一个真实联调 **Bug**。请先定位根因，不要直接给表象修复。

现象：

- 用户操作步骤
- 预期结果
- 实际结果
- 前端可见提示
- 后端响应码和响应体

上下文：

- 相关 **Controller / Service / Repository** 代码
- 相关数据库表当前记录
- 涉及的外部系统状态，例如 **Gitea PR state / merged**
- 哪些接口会触发同步，哪些接口只读本地表

约束：

- 修复必须在服务端保证一致性，前端提示只能作为补充。
- 需要覆盖重复提交、旧数据、未登录、越权、状态过期场景。
- 输出时区分“根因修复”和“用户体验补充”。

4.4 下一阶段计划改进

1. 让 AI 在生成代码前必须输出“状态机检查表”和“权限检查表”。
2. 让 AI 在修 Bug 前必须回答“这个修复是否只处理表象”。
3. 对跨模块问题，要求 AI 画出调用链或列出所有读写路径。
4. 对外部系统同步问题，要求 AI 区分“远端真实状态”“本地缓存状态”和“前端展示状态”。
5. 所有 AI 生成代码必须配套至少一个失败用例和一个成功用例，否则不进入合并。

5. 本阶段的核心工程判断

5.1 决策内容

本阶段最关键的工程判断是：Git Guild 的 P4 演示版本必须以“真实核心链路可运行”为优先，而不是把所有 P2 展示功能都做满。

团队最终保留并优先打通：

- 用户注册、登录、JWT 与角色路由。
- Gitea 仓库导入、Issue 同步、PR 同步。
- 委托发布、管理员审核、冒险家接取。
- 成果提交、维护者审核、PR 合并校验。
- XP 发放、成长档案基础展示。
- 单元测试、集成测试、CI/CD 和 Bug 修复日志。

团队有意识降级或延后的内容：

- 推荐匹配只保留轻量实现，不作为核心演示阻塞项。
- 徽章以基础展示为主，不追求完整规则体系。
- 高级通知、邮件、复杂 mentor 模式延后。

5.2 为什么 AI 无法单独做这个决策

AI 可以根据需求列表建议优先级，但无法单独判断以下工程现实：

- 课程验收更看重哪条演示链路。
- 团队实际剩余时间和每个成员的熟悉模块。
- 哪些功能虽然“好看”，但一旦出错会拖垮主流程。
- 哪些 Bug 必须修复，哪些可以通过演示说明规避。

例如 AI 可能建议补推荐、徽章、统计图等展示功能，但团队判断 P4 最重要的是仓库接入、任务发布、接取、提交、审核、成长闭环。只要这个闭环不稳，其他展示功能再完整也无法支撑验收。

5.3 团队最终如何判断和取舍

团队采用三条标准取舍：

标准	说明	决策结果
是否阻塞主演示链路	登录、发布、审核、接取、提交、PR、成长任一断点都会影响验收	必修
是否能被测试稳定覆盖	能用单元测试或集成测试覆盖的核心逻辑优先做扎实	必修
是否只是增强展示	不影响主链路的 P2 功能可以保留轻量实现或延后	可降级

最终取舍是：优先把 P0/P1 核心链路做成可验证、可回归、可演示的版本；P2 功能只做不影响稳定性的展示增强；所有影响演示的 Bug 必须进入 Bug 修复日志并完成验证。